L

LOFT MOTION
browser-based motion design engine

Loft Motion × Claude

Turn a sentence into motion.

A practical guide to wiring Claude into Loft Motion so it can author animations from a text prompt and hand back an editable project or a finished video.

THREE WAYS TO CONNECT

- MCP server (Claude Desktop)
- Paste in the app
- Automated URL / API

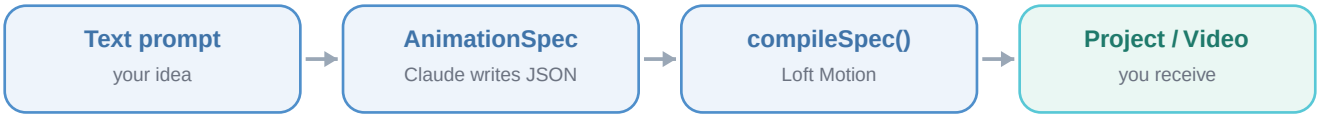
Live app: **fanciful-bonbon-5ab3b6.netlify.app**

Full reference in the repo: docs/LLM_AGENT.md

How it works

Loft Motion is a motion-design engine whose single source of truth is one validated JSON “scene document.” Anything that can produce a valid scene can drive the whole engine — the renderer, the timeline, and every exporter.

Rather than make Claude hand-write hundreds of keyframes, you give it a small, intent-level format called an AnimationSpec. Claude describes WHAT should happen (“a title that rises, a glowing orb that pulses”); a compiler expands that into a full, beautiful, valid scene — adding the keyframes, staggers and easings for you.



What you can get back

- A Loft Motion project (.loft.json) — opens in the app to keep editing.
- A rendered video — MP4, GIF, or transparent WebM.
- A web/vector export — Lottie JSON, standalone CSS, or a React component.

RULE OF THUMB

Want to keep editing it? Ask for the project. Want a file to post or embed? Ask for a video (or Lottie/CSS for the web). The app reports exactly what survives each format — e.g. glow effects only render in MP4.

Path A · MCP server

Best for chat with Claude (Claude Desktop, or any MCP client). Claude calls Loft Motion as a set of tools.

Setup (once)

1. **Install dependencies**
   In the project folder, run npm install. That is all the agent tooling needs — there is no build step.

2. **Register the server with Claude Desktop**
   Add this to claude_desktop_config.json, using the absolute path to the file, then restart Claude Desktop.

```
{
  "mcpServers": {
    "loft-motion": {
      "command": "node",
      "args": ["/abs/path/to/Loft-Motion/mcp-server/loft-motion-mcp.mjs"]
    }
  }
}
```

3. **Talk to Claude in plain English**
   For example:

> “Use Loft Motion to make a 6-second square teaser: a glowing orb that pulses in the center, the title ‘Ship It’ rising above it, and a subtitle fading up below. Save the project.”

What Claude does for you

- Calls get_guide to learn the format, then writes an AnimationSpec.
- Calls create_project to compile it into a .loft.json (it can write the file to a path you give).
- If a spec is invalid, the tool returns the exact error and Claude self-corrects.

Tools the server exposes

```
get_guide            # the authoring guide (Claude reads this first)
get_spec_schema      # JSON Schema for an AnimationSpec
get_example_spec     # a sample spec to adapt
describe_vocabulary  # every preset name the spec accepts
create_project       # compile a spec -> a .loft.json project
```

Path B · Paste in the app

Zero setup, most visual — works with ANY Claude (web, desktop, API).

1. **Open the app and click “Prompt”**
   In the toolbar (sparkle icon, next to Examples). Click “Copy authoring guide.”
2. **Ask Claude for a spec**
   Paste the guide to Claude, then add your request — e.g. “now write an AnimationSpec for a product-launch title that pops in with a glow.”
3. **Build it**
   Paste the JSON Claude returns into the Prompt panel and click “Build animation.” It appears live on the stage.
4. **Export**
   Scrub and tweak, then open Export… and choose Project (.loft.json) or a video (MP4 / GIF / WebM / Lottie / CSS).

Path C · Command line (scripted)

Good for batch jobs or piping Claude’s output straight to a file.

```
npm run agent guide          # print the authoring guide for Claude
npm run agent example        # a sample spec to copy

# Claude writes spec.json, then:
npm run agent build spec.json -o teaser.loft.json

# ...or pipe a spec straight in:
echo '{"elements":[{"type":"text","text":"Hi","enter":"pop"}]}' \
  | npm run agent build -o hi.loft.json
```

SELF-CORRECTING
Invalid specs print a precise, path-pointed error (e.g. elements.0.text: expected string) and exit non-zero — so a script or agent loop can read it and retry.

Path D · Automated video

Loft Motion renders and encodes video in the browser, so to get an MP4 with no clicks, point a browser (or a headless agent) at the live app with the spec in the URL. The page loads it, renders, and downloads the file by itself.

```
https://fanciful-bonbon-5ab3b6.netlify.app/?spec=<encoded-json>&export=mp4&download=1

# other formats: export=gif | webm | lottie | css | sprite | project
# add &autoplay=1 to preview;  put big payloads in the URL #hash
```

BROWSER NOTE

MP4 needs the WebCodecs encoder — use Chrome, Edge, or Firefox 130+. Lottie / CSS / GIF work everywhere.

Or call the page API directly (computer-use / Playwright)

Every loaded page exposes a small global so an agent driving a real browser can author and export by calling functions instead of clicking:

```
await LoftAgent.load(spec)            // build a spec onto the stage
await LoftAgent.exportVideo('mp4')    // render + download a video
await LoftAgent.saveProject()         // download the .loft.json
LoftAgent.guide                       // the authoring guide (string)
LoftAgent.schema()                    // AnimationSpec JSON Schema
```

AnimationSpec cheat sheet

Claude describes intent with named presets — never raw keyframes. Two element types: text and shape.

Placement (at)

`center` `title` `subtitle` `top` `bottom` `left` `right` `top-left` `top-right` `bottom-left` `bottom-right` `lower-third` `{x,y}`

Entrances (enter)

`fade` `fade-up` `fade-down` `from-left` `from-right` `from-top` `from-bottom` `pop` `scale` `zoom` `rise` `spin` `blur-in`

Exits (exit)

`fade` `to-left` `to-right` `to-top` `to-bottom` `pop` `scale` `zoom` `drop` `spin`

Emphasis (continuous life)

`float` `drift` `pulse` `spin` `orbit` `sway` `wiggle`

Effects

`glow` `soft-glow` `shadow` `blur` `vignette` `grain` `chroma` `duotone` `outline`

Sizes

`1080p` `4k` `square` `story` `portrait` `wide` `banner` `thumbnail` `{width,height}`

TOP-LEVEL FIELDS

title, size, fps, duration (inferred if omitted), background (hex or “transparent”), palette[], stagger (cascades auto-started elements), and elements[].

A worked example

Prompt to Claude:

> “Make a 5-second square intro: a glowing orb that zooms in and gently pulses in the center, with the title ‘Hello, Loft’ rising above it.”

Claude returns this AnimationSpec:

```
{
  "title": "Hello Loft",
  "size": "square",
  "duration": 5,
  "background": "#0b0d12",
  "stagger": 0.12,
  "elements": [
    { "type": "shape", "shape": "circle", "size": 340, "at": "center",
      "color": "#1b2a4a", "gradientTo": "#4f8fcb",
      "enter": "zoom", "effects": "soft-glow", "emphasis": "pulse" },
    { "type": "text", "text": "Hello, Loft", "at": "title",
      "fontSize": 150, "weight": 800, "enter": "rise", "effects": "glow" }
  ]
}
```

Then you deliver it — pick one:

- Project: create_project (MCP), npm run agent build, or Export -> Save project.
- Video: open the app with ?spec=…&export=mp4&download=1, or LoftAgent.exportVideo('mp4').

Quick reference

Commands

```
npm install                              # one-time
npm run dev                              # run the app locally (localhost:3000)
npm run mcp                              # start the MCP server
npm run agent guide|schema|example       # the LLM contract
npm run agent build spec.json -o out.loft.json
```

Files worth knowing

```
mcp-server/loft-motion-mcp.mjs   # the MCP server (zero dependencies)
scripts/loft-agent.mjs           # the CLI
src/lib/agent/                   # spec + compiler (the engine of this)
docs/LLM_AGENT.md                # the full reference
```

Choosing a deliverable

- Edit / iterate / hand off a source file -> project (.loft.json)
- Post or embed a clip -> MP4 / GIF / WebM
- Lightweight web animation -> Lottie or CSS (note: shader effects are MP4-only)

Live app: **https://fanciful-bonbon-5ab3b6.netlify.app/**

Tip: the in-app Prompt panel has one-click buttons to copy the guide and JSON Schema straight to Claude.